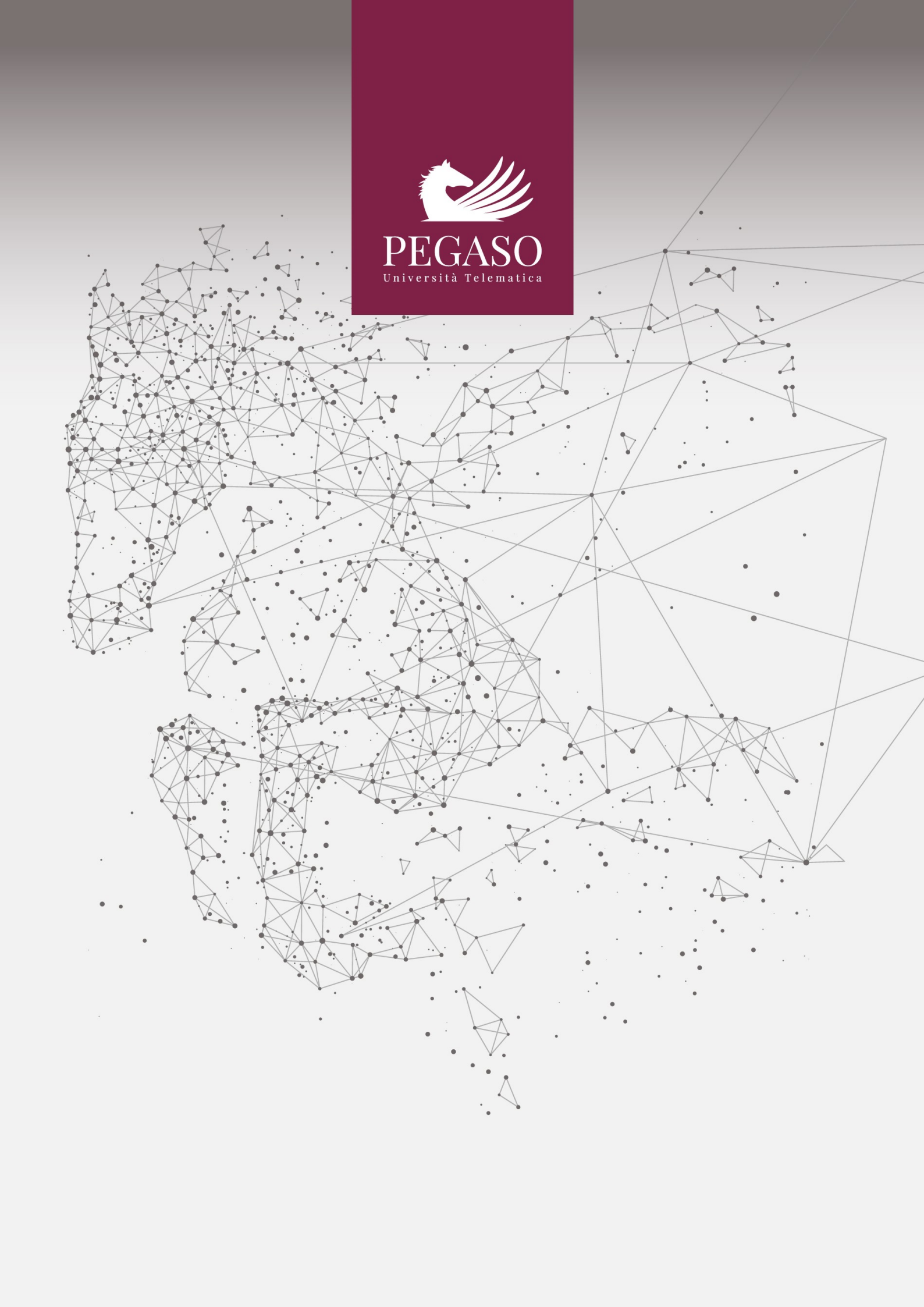

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. CONCETTO DI ECCEZIONE 4

3. GESTIONE DELLE ECCEZIONI 6

3.1 GESTIONE DI PIÙ TIPI DI ECCEZIONE 8

4. CLAUSOLE NELLA GESTIONE DELLE ECCEZIONI 10

4.1 CLAUSOLE ELSE 10

4.2 CLAUSOLE FINALLY 11

BIBLIOGRAFIA 12

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introduce il concetto di eccezione. Verranno presentate le strategie sia per evitare le eccezioni, sia per gestirle. Infine, verranno illustrate le modalità di gestione di più tipi di eccezioni contemporaneamente e le clausole messe a disposizione da Python per una gestione più accurata delle eccezioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Concetto di eccezione

Un'**eccezione** è un errore che si verifica a run-time, cioè durante l'esecuzione di un programma. Solitamente un'eccezione interrompe in modo brusco il programma.

Nel caso di eccezione, Python visualizza a schermo il **traceback**, cioè un messaggio di errore che fornisce informazioni sul numero della linea del programma (istruzione) che ha causato l'eccezione. Nel messaggio viene indicato anche il tipo di eccezione ed una breve descrizione dell'errore come nell'esempio seguente:

```
Traceback (most recent call last):
  File "C:\    ve.py", line 4,
 in <module>
    print(a/b)
ZeroDivisionError: division by zero
```

I programmi non possono rilevare in autonomia la differenza tra dati corretti e dati errati se non previsto dal programmatore. Se l'utente inserisce dei dati errati, infatti, il programma produrrà in output dati errati. In questo caso si verifica il cosiddetto **fenomeno GIGO: garbage in, garbage out** (letteralmente "spazzatura in ingresso, spazzatura in uscita" riferita ai dati).

È importante, quindi, progettare un programma in modo tale che i dati di input errati non siano accettati.

Generalmente le eccezioni si possono evitare con la **programmazione** (uso di strutture decisionali o iterative).

Ad esempio, il **ciclo di validazione dell'input** è una delle tecniche per evitare di considerare input errati. Con questa tecnica, l'inserimento dell'**input viene ripetuto finché non è corretto** (non noto il numero di iterazioni che comunque sarà maggiore o uguale a 0).

La condizione della struttura iterativa che implementa il ciclo di validazione dell'input prevede, solitamente, l'uguaglianza della variabile da controllare con il valore errato. Iterando per condizione vera, infatti, si continuerà a chiedere in input un valore finché esso non è corretto.

Dunque, il ciclo di validazione dell'input verifica i dati di input prima che siano processati dal programma:

- se i dati di input sono errati, viene mostrato un messaggio di errore e si richiedono i dati;
- altrimenti tali dati vengono elaborati.

Si ipotizzi di voler evitare una divisione per zero tra due numeri raccolti come input dall'utente. Dopo aver raccolto il primo numero, bisogna far sì che il secondo numero sia diverso da zero. Il diagramma di flusso che permette di risolvere tale problema è riportato in Figura 1.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

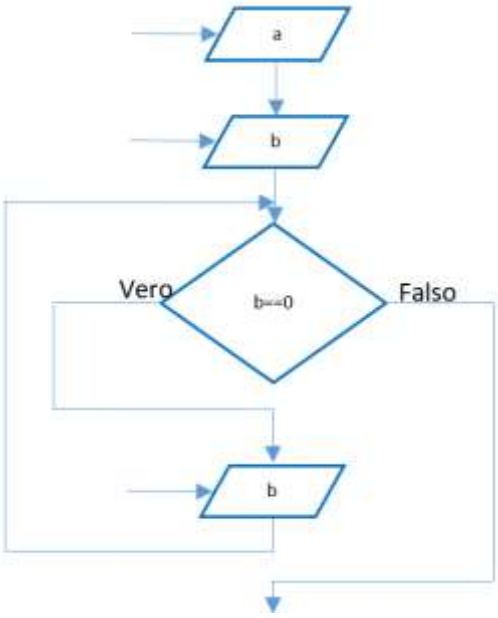


Figura 1. Diagramma di flusso del ciclo di validazione dell'input per evitare una divisione per zero

Il codice Python organizzato in un'unica funzione main che risolve il problema della divisione per zero è il seguente:

```
def main():
    a = int(input("Inserire a: "))
    b = int(input("Inserire b: "))

    while b==0:
        b = int(input("Inserire b: "))

    print(a/float(b))

main()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Gestione delle eccezioni

Alcune eccezioni però non possono essere evitate in fase di coding come ad esempio:

- conversione di una stringa non numerica in un intero;
- apertura di un file in lettura e il file non esiste.

In questi casi è necessario l'utilizzo di un **gestore delle eccezioni** (**exception handler**). Il gestore delle eccezioni è una porzione di codice (blocco di istruzioni) che viene eseguito quando viene sollevata un'eccezione evitando l'interruzione anomala del programma.

A tal proposito, Python mette a disposizione l'istruzione **try/except** il cui formato generale è:

```
try:
    istruzioni
except tipo_eccezione:
    istruzioni
```

Nel blocco di istruzioni che seguono alla **keyword try** si inseriscono quelle istruzioni che possono potenzialmente sollevare un'eccezione, mentre il gestore delle eccezioni è contenuto nel blocco istruzioni dopo la keyword **except**.

Se l'istruzione nella clausola try solleva un'eccezione:

- Se il tipo di eccezione sollevata è quello specificato nella clausola except:
  - gestore fa in modo tale che vengano eseguite le istruzioni del blocco except;
  - si continua con il programma subito dopo l'istruzione try/except.
- Se l'eccezione sollevata non è quella specificata nella clausola except il programma si interrompe con un messaggio di errore traceback.

  Se non viene sollevata nessuna eccezione, invece, il codice del gestore delle eccezioni viene saltato.

Si mostrano ora due esempi di eccezione. Il primo è un'eccezione di tipo aritmetico: **divisione per zero**. Il codice sorgente corrispondente a tale eccezione potrebbe essere il seguente:

```
a = 0
b = 0
print(a/b)
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

che genera il seguente traceback:

```
Traceback (most recent call last):
  File "C:\    ve.py", line 4,
 in <module>
    print(a/b)
ZeroDivisionError: division by zero
```

Il codice che gestisce l'eccezione precedente e non permette la terminazione anomala del programma è il seguente:

```
a = 3
b = 0

try:
    print(a/b)
except ZeroDivisionError:
    print("Attenzione: divisione per zero")
```

che visualizza a schermo il seguente messaggio:

```
Attenzione: divisione per zero
```

Il secondo esempio è un'eccezione inerente i file: **apertura in lettura di un file inesistente**. Il codice sorgente corrispondente a tale eccezione potrebbe essere il seguente:

```
infile = open("ciao.txt",'r')
```

che genera il seguente traceback:

```
Traceback (most recent call last):
  File "C:\ ove.py", line 1,
 in <module>
    infile = open("ciao.txt",'r')
FileNotFoundError: [Errno 2] No such file or directory: 'ciao.txt'
```

Il codice che gestisce l'eccezione precedente e non permette la terminazione anomala del programma è il seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
try:
    infile = open("ciao.txt",'r')
except FileNotFoundError:
    print("File inesistente")
```

che visualizza a schermo il seguente messaggio:

```
File inesistente
```

3.1 Gestione di più tipi di eccezione

Spesso il codice inserito nel blocco try può sollevare più di un tipo di eccezione. C'è bisogno, quindi, di scrivere una except per ciascun tipo di eccezione che è necessario gestire. Una except che non indica una specifica eccezione gestirà qualsiasi eccezione sollevata nella try.

Ovviamente, la gestione avverrà in modo generico senza sapere qual è l'effettiva eccezione che è stata sollevata. La clausola except senza l'indicazione del tipo di eccezione gestita deve essere l'ultima della serie dei gestori.

Un esempio di codice sorgente in Python per la gestione di più tipi di gestione è il seguente:

```
def main():
    try:
        infile = open("votazioni.txt",'r')

        totale = 0
        contatore = 0

        for riga in infile:
            voto = int(riga.rstrip('\n'))
            totale = totale + voto
            contatore = contatore +1

        print("La media dei voti è ", float(totale)/contatore)

    except ZeroDivisionError:
        print("Nessuna votazione disponibile")

    except FileNotFoundError:
        print("File inesistente")

    except ValueError:
        print("Votazione non numerica")

    except:
        print("Si è verificato un errore")

main()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice appena presentato gestisce:

- l'eccezione relativa alla divisione per zero;
- l'eccezione relativa all'apertura in lettura di un file inesistente;
- l'eccezione relativa all'inserimento in input di un valore non numerico di cui si chiede la conversione in numero;
- tutti gli altri tipi di eccezioni in modo generico.

È possibile, inoltre, utilizzare un **oggetto eccezione** che viene creato in memoria nel momento in cui l'eccezione viene sollevata. Solitamente, tale oggetto contiene un messaggio di errore di default relativo all'eccezione stessa. È possibile assegnare ad una variabile l'oggetto eccezione nella clausola except:

```
except ValueError as errore:
```

La variabile dell'oggetto eccezione può essere passata alla funzione print come argomento per mostrare il messaggio di errore di default:

```
print(errore)
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Clausole nella gestione delle eccezioni

4.1 Clausole else

L'istruzione **try/except** può includere un **else opzionale** che appare dopo tutte le except.

L'else opzionale deve essere allineato con try e except e presenta una sintassi simile all'else dei costrutti di selezione. In particolare, l'else opzionale è un blocco di istruzioni eseguite dopo le istruzioni nella try solo se nessuna eccezione viene sollevata. Infatti, se viene sollevata un'eccezione, il blocco relativo all'else viene saltato.

Si ipotizzi di dover calcolare la media dei voti letti da un file di testo. Le eccezioni che potrebbero essere sollevate sono diverse:

- errore nel nome fisico e/o nel path del file;
- file non esistente;
- uno o più valori contenuti nel file non numerici;
- file esistente, ma vuoto senza valori (media non calcolabile poiché contatore dei libri a zero e conseguente divisione per zero).

Ai fini didattici, viene riportato di seguito il codice sorgente che gestisce genericamente le eccezioni con la visualizzazione del messaggio di default dell'oggetto eccezione e utilizza l'else opzionale per la stampa della media solo nel caso in cui nessuna eccezione viene sollevata.

```
def main():
    try:
        infile = open("votazioni.txt",'r')

        totale = 0
        contatore = 0

        for riga in infile:
            voto = int(riga.rstrip('\n'))
            totale = totale + voto
            contatore = contatore +1
        media = float(totale)/contatore

    except Exception as errore:
        print(errore)
    else:
        print("La media dei voti è ",format(media,'.2f'))

main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4.2 Clausole finally

I blocchi try/except possono includere anche una **clausola finally** che compare dopo tutti gli except. Finally deve essere allineato con try e except e presenta il seguente formato generale:

```
finally:
    istruzioni
```

Il blocco istruzioni contenuto nella finally viene eseguito sia nel caso in cui l'eccezione venga sollevata, sia nel caso in cui non venga sollevata. L'obiettivo delle istruzioni di questo blocco è quella di "sistemare" tutto prima di uscire (es. chiusura file).

Si riporta il codice sorgente dell'esempio precedente di media di votazioni contenute in un file di testo a cui viene aggiunta la clausola finally dopo l'else opzionale:

```
def main():
    try:
        infile = open("votazioni.txt",'r')

        totale = 0
        contatore = 0

        for riga in infile:
            voto = int(riga.rstrip('\n'))
            totale = totale + voto
            contatore = contatore +1
        media = float(totale)/contatore

    except Exception as errore:
        print(errore)
    else:
        print("La media dei voti è ",format(media,'.2f'))

    finally:
        infile.close()

main()
```

Se non ci sono except che specificano eccezioni del tipo giusto oppure se l'eccezione viene sollevata al di fuori di un blocco try (entrambi rappresentano casi di eccezioni non gestite), l'eccezione interromperà bruscamente il programma.

La documentazione di Python fornisce le informazioni circa le eccezioni che possono essere sollevate dalle diverse funzioni e sono disponibili al seguente link: https://docs.python.org/3/library/exceptions.html#bltin-exceptions

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).